University of Geneva

Department of Astronomy

MSc Astrophysics

Astrophysics Laboratory II

Orbit and Attitude Analysis Tool for the CHEOPS Space Telescope

Supervised by:

Dr. Alexis Heitzmann

Dr. Adrien Deline

Student: Feras Natsheh

Spring Semester

2025–2026 Academic Year



UNIVERSITÉ DE GENÈVE

FACULTÉ DES SCIENCES

Département d'astronomie

Contents

1 Introduction **3**

2 An Overview of CHEOPS **4**
2.1 Spacecraft Orbit 4
2.2 The Roll Angle and Its Consequences 4
2.3 Spacecraft Body Frame 6
2.4 Inertial Reference Frame 8

3 Methods **9**
3.1 Used Data Products 9
3.2 Quaternion-Based Attitude Representation 10
3.3 Angular Computations 11
3.3.1 Sun & Moon 11
3.3.2 Earth Center and Limb 11
3.4 Cosmographia Input Generation 12

4 Results **13**
4.1 High-Jitter Visits 15
4.2 Anomalies 17
4.3 3D Orbit Visualization with Cosmographia 18

5 Limitations and Future Extensions **18**
5.1 Limitation on the Current Implementation 18
5.2 Future Extensions 19

6 Conclusion **20**

A Software Architecture & Design **21**
A.1 Overview 21
A.2 Constants 21
A.3 Data Loader 22
A.4 Plot Architecture & Interactive Legends 22
A.5 Single Timestamp vs. Time-Range 23

B Command-Line Interface **23**
B.1 Flags & Arguments 23
B.2 Examples 23

C Data File Formats & Naming Conventions **24**
C.1 FITS Files 24
C.2 XML Files 24

C.3 Time Systems 25

D Dependencies and Used Libraries **25**

E Quaternion Convention & Pipeline **26**

E.1 FITS Convention 26

E.2 Cosmographia Convention 26

F Cosmographia Visualization Guide **26**

F.1 Overview 26

F.2 Requirements 26

F.3 File Converter 27

F.4 Loading into Cosmographia 27

Abstract

This report presents `whereischeops_v0.0.py`, a Python command-line utility for querying, visualizing, and analyzing the position and spacecraft orientation of the CHEOPS space telescope at a given time. CHEOPS is a space telescope dedicated to the characterization of exoplanets. At the Observatory of Geneva, the CHEOPS Science Operations Center (SOC) is responsible for all science-related operations of the mission, including observation scheduling, interactions with observers, data processing, distribution, and long-term archiving. This Python tool is designed to aid the daily tasks of the CHEOPS SOC team, and will support the investigation of unexpected events that may occur during operations, such as spacecraft pointing deviations or on-board temperature variability. In addition, we developed complementary 3D visualizations of CHEOPS in low-Earth orbit using NASA's Cosmographia Visualization Tool, which help diagnose the spacecraft's orientation, validate the results from the `whereischeops` tool, and generate 3D visualizations of the spacecraft for communication and public outreach.

1 Introduction

The **CHaracterising ExOPlanet Satellite** (CHEOPS) is a joint mission between ESA and Switzerland with important contributions by ten additional ESA Member States dedicated to high-precision optical photometry of bright stars already known to host transiting exoplanets (Benz et al., 2021). Launched on 18 December 2019 from the Guiana Space Centre in Kourou, French Guiana, CHEOPS occupies a low Earth orbit (LEO) at an altitude of approximately 700 km in a sun-synchronous configuration with an orbital period of roughly 99 minutes. Its primary science instrument is an F/8 Ritchey–Chrétien telescope with a 30 cm effective aperture, delivering photometric precision at the level of tens of parts per million per hour for bright targets.

The tight photometric requirements demand a great attitude stability. Any pointing error or jitter that exceeds the instrument's tolerance can introduce systematic noise and potentially corrupt the data. Pointing jitter occurs due to different reasons such as blinding of the star trackers, fast rolling of the spacecraft, and more. Understanding when jitter occurs, and what drives it, requires inspection of the spacecraft's geometric state, and the orientation of CHEOPS' components (star-tracker heads, solar panels, radiators, and telescope) with respect to the Sun, Moon, and Earth at the time of each case.

This report describes `whereischeops_v0.0`*, a Python command-line utility that:

(i) loads all relevant CHEOPS data products for a specified time interval

(ii) constructs a uniform time grid and computes the inertial-frame directions of the main spacecraft body axes via quaternion rotation

*`https://github.com/ferasnatsheh/whereischeops`

(iii) calculates and plots the angular separation of each axis to the Sun, Moon, and Earth center as a function of time

(iv) auto-detects pointing-jitter data and produces a two-panel diagnostic plot correlating the jitter time series with the body axes, and

(v) supports interactive legend toggling and is designed to scale a large archival dataset.

This report is structured as follows: Sections 1 through 6 present the scientific context, methods, validation, and results, forming the APL II submission for assessment. Appendices A through F constitute technical documentation intended for the CHEOPS Science Operations Center and future users of the `whereischeops` script, covering the full software architecture, command-line interface, data format details, and a step-by-step guide for the Cosmographia visualizations.

2 An Overview of CHEOPS

2.1 Spacecraft Orbit

CHEOPS follows a sun-synchronous orbit at an altitude of $h \approx 700$ km above Earth's surface, corresponding to a geocentric distance of $|\mathbf{r}| \approx 7071$ km. The orbital period is $T \approx 99$ minutes, meaning that a typical CHEOPS observation spanning several hours covers multiple complete orbits. As in all LEO observations, the science duty cycle is modulated by Earth occultations, during which the target passes behind the terrestrial disk, passages through the South Atlantic Anomaly (SAA), during which elevated particle flux introduces higher noise, and stray-light events, during which the line of sight grazes the illuminated limb of the atmosphere. Together, these three effects can account for a substantial fraction of each orbit, and their cumulative impact on the science duty cycle depends on the target's sky position and the orbital geometry at the time of the observation.

The sun-synchronous orbit is necessary for the thermal stability of CHEOPS, and in turn, the photometric precision of the detector. By keeping the orbital plane closely aligned with the terminator, the spacecraft maintains a consistent angle between its solar panels and the Sun throughout the year, which stabilizes the thermal environment of the instrument. Similarly, the optical telescope of CHEOPS is deliberately defocused so that the starlight is spread across a broader, more stable point-spread function (PSF) rather than concentrated near the diffraction limit. This trades angular resolution for a PSF shape that is less sensitive to small pointing deviations which are caused by saturation for bright targets and flat-field corrections (Benz et al., 2021).

2.2 The Roll Angle and Its Consequences

The geometric challenge at the heart of this project arises from a set of constraints that CHEOPS must satisfy simultaneously during every observation. The telescope line of sight (LoS) must

remain fixed on the science target, the solar panels must remain illuminated to maintain power, the radiators must point away from both the Earth and the Sun in order to efficiently cool the instrument, and the LoS must never point within a minimum angular distance of the Sun, set by the Sun Exclusion Angle (SEA) of 115°, to prevent solar stray light from entering the telescope tube and contaminating or damaging the detector. The SEA is defined exclusively for the telescope LoS, but as a consequence, it imposes constraints on other components of the spacecraft.

These requirements are satisfied by the continuous rotation of the spacecraft about its LoS. Since the LoS is fixed to the science target in inertial space, this rotation corresponds to a continuous spin of the entire spacecraft body about that fixed axis as the orbital geometry evolves over each 99-minute orbit. This rotation is called the roll angle. It is a necessary consequence of the geometric constraints listed above, and it evolves continuously, though not necessarily linearly, throughout each orbit. Its value at any given instant depends on the spacecraft's position along the orbit and the target's right ascension and declination, making it a time series rather than a fixed parameter of the observation. Figure 1 illustrates how this roll manifests on the detector. As the spacecraft rotates about the LoS, field stars trace small arcs around the fixed science target rather than remaining at constant pixel coordinates. The PSF itself does not rotate, but the changing positions of background stars add complexity to photometric extraction.

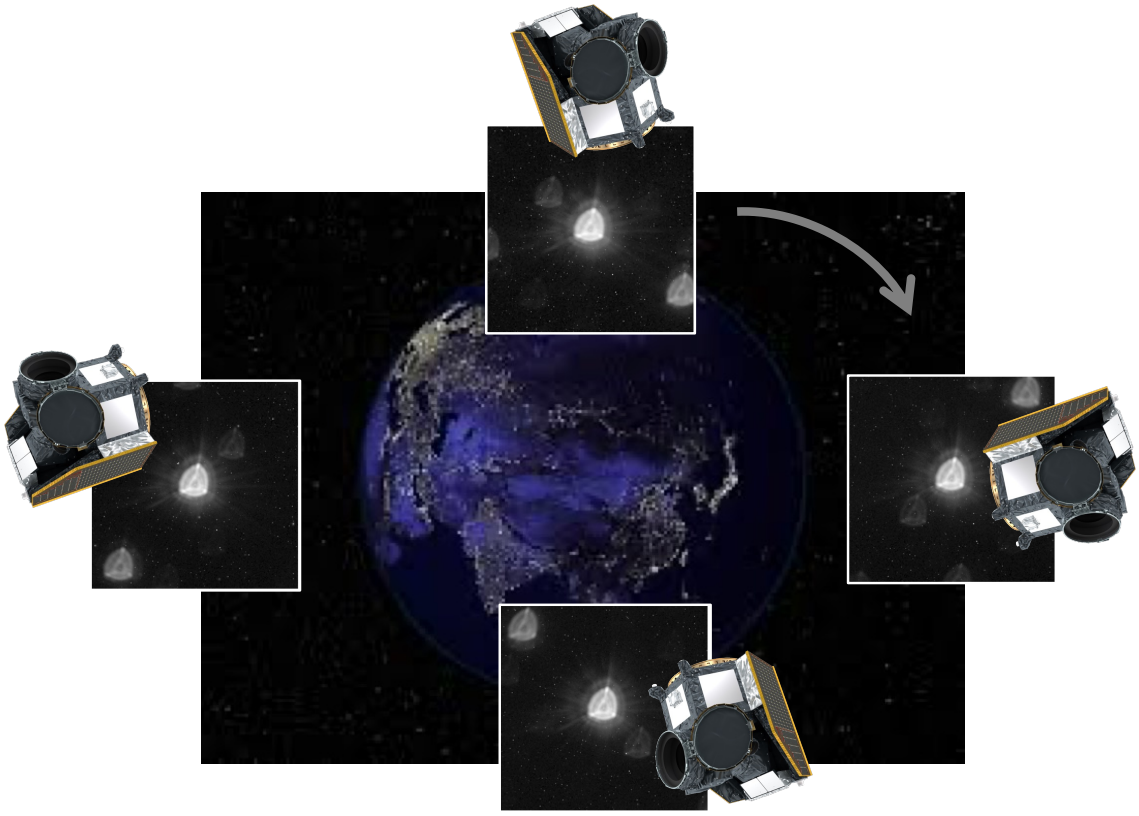


Figure 1: Adopted from Fortier et al. (2024). The spacecraft is nadir-locked to ensure thermal stability. As a result, the rotation of CHEOPS around Earth is reflected in the images, where the stars in the field of view rotate around the pointing direction.

Since the roll is a rotation about the LoS, the LoS in the inertial reference frame is unaffected, but other components of the spacecraft are swept continuously through different orientations relative to the Sun, Earth, and Moon as the orbit progresses. These components include the radiators, solar panels, and the Star Tracker Optical Heads (STOHs). CHEOPS has two STOHs, positioned 90° apart (each tilted 45° from the $+Z$ axis, as shown in Table 1), and their primary function is to trace the star field within the field of view and match detected stars against an onboard catalog. The resulting orientation measurements are fed into the Attitude and Orbit Control System (AOCS), which uses them as the primary reference for maintaining the telescope on target (Benz et al., 2021). When the Sun approaches close to a STOH's field of view, scattered solar photons saturate the background in the star tracker image and reduce the number of stars the head can reliably detect. With fewer valid detections, the AOCS receives a degraded attitude estimate and loses the precision it needs to keep the LoS locked on the science target, resulting in increasing pointing jitter.

This is one of the questions `whereischeops` is designed to help answer: at what point in the orbit do the STOHs approach the Sun most closely, and how does that geometry correlate with observed pointing jitter? This is done by computing the angle from each STOH to the Sun and overlaying the corresponding jitter time series when applicable.

2.3 Spacecraft Body Frame

A coordinate system fixed to the satellite structure, the body frame, defines the physical direction each component of the spacecraft points in. Table 1 lists the conventions adopted here, based on the CHEOPS instrument description in Benz et al. (2021). The $+X$, $+Y$ and $+Z$ body axes are also shown in Figure 2. The LoS points along $+X$, while the $+Z$ axis is the "anti-nadir" direction. The roll is managed through a pointing mode known as nadir locking, in which the spacecraft's $+Z$ body axis is kept as close to the anti-nadir direction as the roll allows, so that the radiators face away from Earth and radiate heat efficiently. However, the radiators are not mounted with their normal purely along $+Z$. Instead, they are also tilted toward $+X$. As a consequence, nadir locking is not strictly fixed but rather oscillates with the roll as the orbit progresses. In addition, the Sun Exclusion Angle (SEA) mentioned earlier is illustrated in Figure 3, along with the Moon exclusion angle, and other constraints set by Earth occultations, stray light, and passage through the South Atlantic Anomaly.

Axis label	Body-frame vector	Physical meaning
`LoS`	$[1,\ 0,\ 0]$	Telescope line of sight ($+X^{\dagger}$)
`anti_nadir`	$[0,\ 0,\ 1]$	away from Earth ($+Z$)
`radiators`	$[\frac{\sqrt{3}}{2},\ 0, \frac{1}{2}]$	Radiators
`solar_panel_l`	$[-\cos(67°),\ \cos(23°),\ 0]$	Left solar panel
`solar_panel_r`	$[-\cos(67°),\ -\cos(23°),\ 0]$	Right solar panel
`STOH_L`	$[0,\ \frac{1}{\sqrt{2}},\ \frac{1}{\sqrt{2}}]$	Star Tracker Head 1
`STOH_R`	$[0,\ -\frac{1}{\sqrt{2}},\ \frac{1}{\sqrt{2}}]$	Star Tracker Head 2

Table 1: Spacecraft body-frame axes and their physical meanings. All vectors are unit vectors in the satellite-fixed frame.

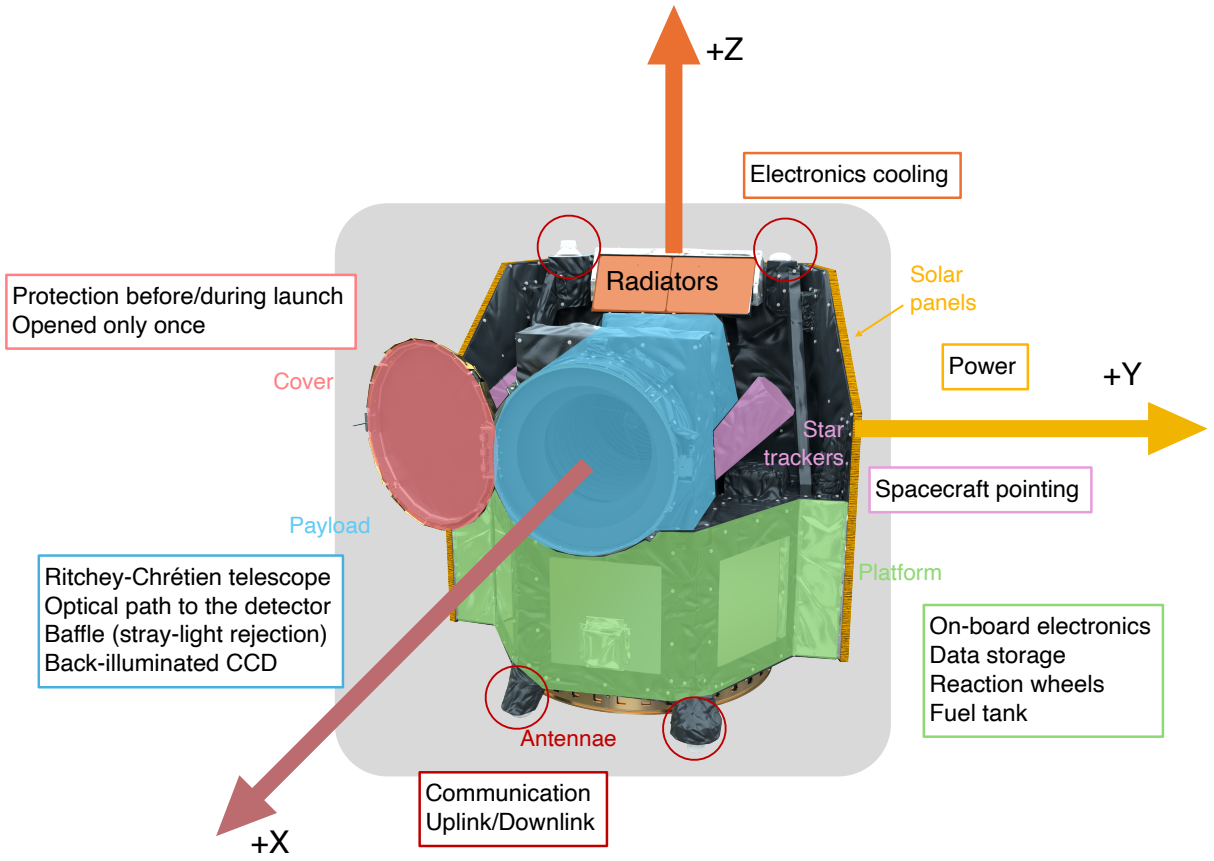


Figure 2: CHEOPS spacecraft configuration and reference frame. Adopted from Dr. Adrien Deline's presentation at the 2026 SIAPS conference with modifications.[‡]

[†]In instances where the target is not at the center of the CCD, the LoS and $+X$ are not the same and have an offset on the order of dozens of pixels. In this case, the AOCS compensates for this and corrects it by fixing the target to the center. This is of the order of 50" and is therefore negligible.

[‡]See: `https://indico.global/event/14632/timetable/?print=1&view=standard_numbered`

[§]See: `https://www.cosmos.esa.int/web/cheops-guest-observers-programme/cheops-observers-manual`

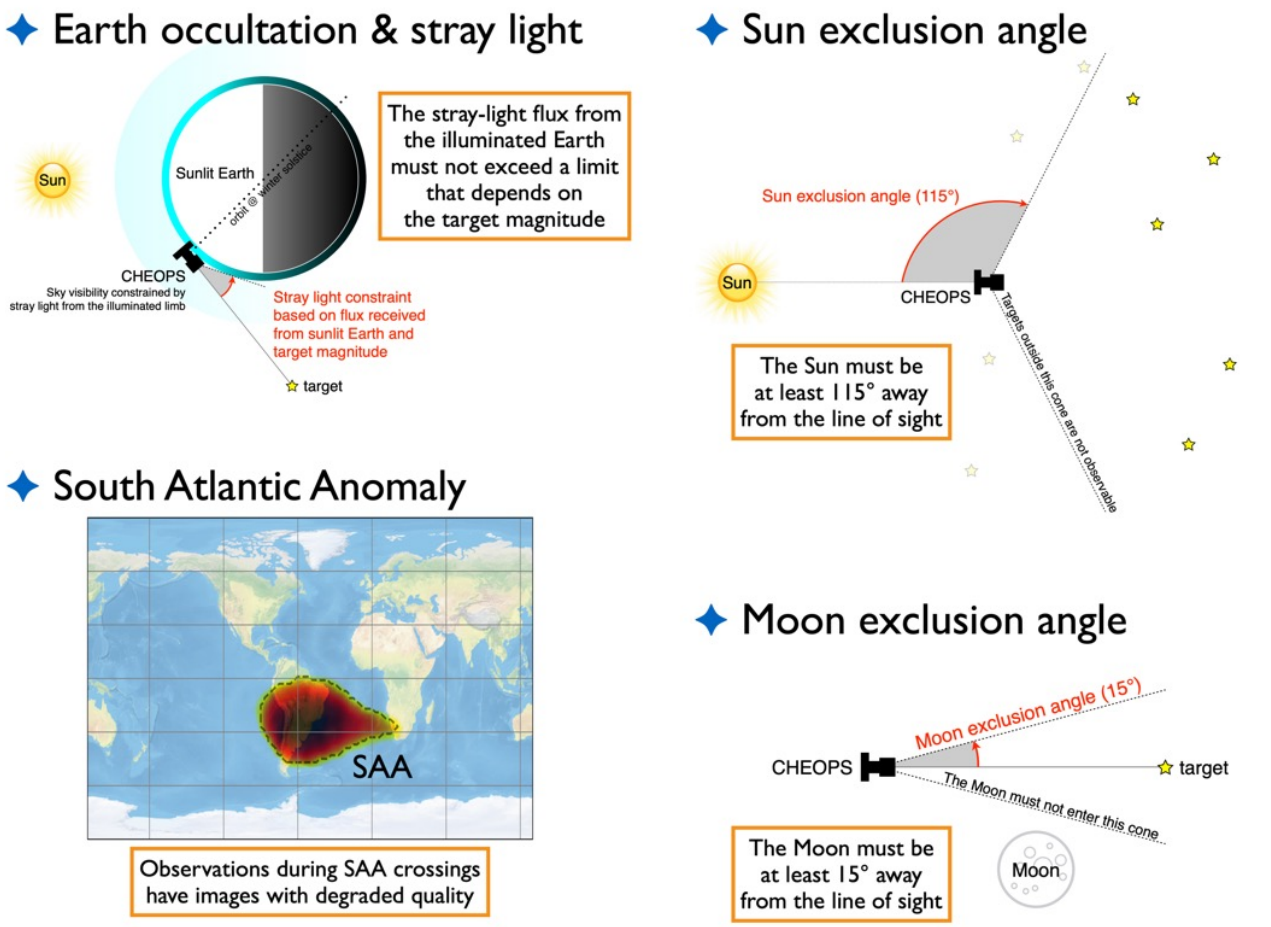


Figure 3: Summary of the conditions for target observability and scientifically valid data acquisition. Figure taken from the CHEOPS Observers Manual.[§]

2.4 Inertial Reference Frame

All spacecraft position vectors from the orbit files are expressed in the Earth Mean Equator and Equinox of J2000.0 frame (EME2000). This is a geocentric, quasi-inertial frame whose axes are fixed by the orientation of Earth's mean equatorial plane and the direction of the mean vernal equinox at the J2000.0 epoch (Kaplan, 2006). The Z-axis points toward the mean celestial north pole, the X-axis toward the mean vernal equinox, and the Y-axis completes the right-handed rule.

Solar-system body positions computed with `astropy.coordinates.get_body` are returned in the Geocentric Celestial Reference System (GCRS), which is the geocentric equivalent of the International Celestial Reference System (ICRS). Unlike EME2000, ICRS is defined kinematically with respect to the positions of extragalactic radio sources rather than to the instantaneous orientation of Earth, making it independent of any particular epoch (Kaplan, 2006). The two reference systems differ by a rotation of a few milliarcseconds, arising from the difference between the mean pole and equinox of J2000.0 and the pole and equinox as defined by ICRS conventions. Therefore, the ICRS is considered to be a good approximation to the J2000.0 frame (Feissel & Mignard, 1998).

For the purposes of this project, we treat EME2000 and GCRS as identical, because the

milliarcsecond-level offset between them is much below the angular precision required for the attitude diagnostics described here.

3 Methods

The geometric state of CHEOPS at any moment in time can be described by two independent quantities: the position of the spacecraft in its orbit, and the orientation of its body frame relative to the inertial reference frame. The goal of the methods described here is to combine these two quantities with the known positions of the Sun, Moon, and Earth in order to compute, at every time step of an observation, the angular separation between each spacecraft body axis and each celestial target.

3.1 Used Data Products

Three categories of native CHEOPS data products, produced by the CHEOPS Mission Operations Center (MOC), feed into `whereischeops`. Each is described below in terms of its file naming convention, internal structure, and role in the pipeline.

Orbit OEM XML files: Spacecraft position is obtained from Orbit Ephemeris Message (OEM) files delivered in XML file format. A typical filename follows the pattern

`CH_FDS_ORBRES_OPER_creation_start_epoch.xml`.

Each file contains a series of state vectors giving the spacecraft position (X, Y, Z) in km and velocity $(\dot{X}, \dot{Y}, \dot{Z})$ in km/s, expressed in the EME2000 inertial frame with UTC timestamps, at a nominal cadence of one state vector every five minutes. `whereischeops` interpolates the position components linearly onto its working time grid using `numpy.interp`. This introduces sub-kilometer position errors, which is negligible for all angular computations described here.

Attitude housekeeping: The primary source of spacecraft attitude used by `whereischeops` is the PSE (Payload Stellar Estimator) quaternion housekeeping file, with product identifier `SCI_RAW_HkAsy...`. The PSE is the onboard subsystem that continuously images the star field through the STOHs, matches detected sources against the onboard stellar catalogue, and outputs a quaternion-based estimate of the spacecraft orientation. This file contains seven columns: three timestamp formats (`OBT_TIME`, `UTC_TIME`, `MJD_TIME`), and four quaternion components (`PSE_quaternion_x`, `PSE_quaternion_y`, `PSE_quaternion_z`, `PSE_quaternion_scal`), sampled at approximately one-second cadence and covering the full duration of the visit. All quaternion-based attitude computations in `whereischeops` draw exclusively from this file.

Raw Attitude Files: The pipeline also delivers a reduced attitude file, `SCI_RAW_Attitude`, which contains `SC_RA` and `SC_DEC` (right ascension and declination of the telescope LoS) and

`SC_ROLL_ANGLE`, all at approximately one-second cadence. This file was the starting point during the development of `whereischeops`. Figure 6 plots RA, Dec, and roll angle read directly from this file. In the main analysis, however, `whereischeops` bypasses this file and works with the raw PSE quaternions from `HkAsy` directly. Future versions of the tool will remove the dependency on `SCI_RAW_Attitude` entirely.

Pipeline quality report: When available, the pipeline quality report (`PIP_REP_MultiParameters`) is loaded as an optional overlay. This file contains many parameters, but `whereischeops` only reads in `pointing error`, `pointing error ra`, `pointing error dec`, and `roll angle`, all time-stamped in MJD. The total pointing error is used as the jitter diagnostic overlaid on the attitude plots in the results section. These categories, along with their file conventions, are further documented in Appendix C.

3.2 Quaternion-Based Attitude Representation

To compute the angle between a spacecraft body axis and a celestial target, we need to know, at each moment in time, which direction in the inertial frame each body axis is pointing. The spacecraft body frame is a coordinate system rigidly attached to the spacecraft structure, whose axes are defined by the hardware layout (Table 1). The inertial EME2000 frame has fixed axes tied to the celestial sphere. The spacecraft's attitude is the rotation that maps one frame to the other, and it changes continuously as the spacecraft rolls.

A three-dimensional rotation has three degrees of freedom. The most intuitive representation, three Euler angles (roll, pitch, yaw), suffers from a fundamental problem known as gimbal lock: at certain orientations, two of the three axes become aligned and one degree of freedom is lost, making the representation singular. Rotation matrices avoid this but require nine numbers to encode three degrees of freedom, carry redundancy, and accumulate floating-point drift under repeated multiplication. Unit quaternions, on the other hand, encode any three-dimensional rotation in exactly four numbers and are free from singularities. For a dataset with tens of thousands of attitude samples that must be applied to multiple body-frame vectors at every time step, these properties make quaternions extremely useful and practical.

A unit quaternion $\mathbf{q} = (q_x, q_y, q_z, q_w)$ encodes a rotation of angle θ about a unit axis $\hat{n} = (n_x, n_y, n_z)$ as

$$q_x = n_x \sin\tfrac{\theta}{2}, \quad q_y = n_y \sin\tfrac{\theta}{2}, \quad q_z = n_z \sin\tfrac{\theta}{2}, \quad q_w = \cos\tfrac{\theta}{2}, \tag{1}$$

subject to the unit-norm constraint $q_x^2 + q_y^2 + q_z^2 + q_w^2 = 1$. The scalar component q_w encodes how large the rotation is (a rotation of 0° gives $q_w = 1$ and all vector components zero), while the vector part (q_x, q_y, q_z) encodes the axis direction.

Given a vector $\hat{b}$ expressed in the spacecraft body frame, its direction in the inertial frame is obtained via the sandwich product

$$\mathbf{v}_{\text{inertial}} = \mathbf{q} \otimes \hat{b} \otimes \mathbf{q}^{-1}, \tag{2}$$

where $\otimes$ denotes quaternion multiplication and $\mathbf{q}^{-1} = \mathbf{q}^* = (q_x^*, q_y^*, q_z^*, q_w^*)$ is the conjugate of $\mathbf{q}$, equivalent to the inverse for a unit quaternion, since $\mathbf{q}\mathbf{q}^* = 1$.

In practice, implementing this via Python is quite straightforward as `whereischeops` utilizes `scipy.spatial.transform.Rotation`, which handles the sandwich product internally via its equivalent rotation matrix, so the implementation never computes quaternion products explicitly. The PSE quaternion stored in `SCI_RAW_HkAsy...` uses the scalar-last convention (q_x, q_y, q_z, q_w) and represents the rotation from the spacecraft body frame to the inertial frame. This means that if you express a body-frame direction in body coordinates and apply this rotation, you obtain the same direction expressed in inertial coordinates. `scipy.spatial.transform.Rotation.from_quat()` accepts the scalar-last convention directly, so no component reordering is needed.

In the `whereischeops` tool, the quaternion attitude data is used by two functions. First, `nearest_rotations(jd_q, quat, jd_query)` takes the array of PSE quaternion timestamps, the corresponding $N \times 4$ quaternion array, and a set of query times, and returns a `scipy Rotation` object containing the nearest attitude at each query time. `angle_to_target(rotations, body_vec, target_vecs)` then applies the rotation stack to a body-frame unit vector `body_vec` using the `rotations.apply()` method, which rotates the vector by the attitude sample in the stack. The angular separation between the rotated body-axis direction and each target direction is then computed as

$$\theta_i = \arccos\Big(\hat{v}_i \cdot \hat{r}_i\Big), \tag{3}$$

where $\hat{v}_i$ is the body-axis direction at time step i expressed in the inertial frame, $\hat{r}_i$ is the unit vector toward the target (Sun, Moon, or Earth centre) at the same epoch. A more technical breakdown of this implementation is included in Appendix D

3.3 Angular Computations

With the body-frame axes rotated into the inertial frame by the quaternion stack, the remaining ingredient is the inertial-frame direction toward each celestial target at every time step.

3.3.1 Sun & Moon

The positions of the Sun and Moon in the inertial frame are obtained from `astropy`'s methods `astropy.coordinates.get_sun(t)` and `astropy.coordinates.get_body('moon', t)`, which return the GCRS Cartesian position of each body at an array of `Time` objects. Then, `angle_to_target` computes the angular separation $\theta = \arccos(\hat{v} \cdot \hat{r}_{\text{target}})$ at every time step, as described in Equation (3).

3.3.2 Earth Center and Limb

The state vectors from the orbit file are expressed in the inertial reference frame (EME2000). So, for a given time, the spacecraft's position in orbit is determined via a vector drawn from

Earth's center to the spacecraft. Therefore, the direction from the spacecraft to Earth's center at each time step is just the negated spacecraft position unit vector, $\hat{r}_{\oplus} = -\hat{r}_{\mathrm{SC}}/|\hat{r}_{\mathrm{SC}}|$, where $\hat{r}_{\mathrm{SC}}$ is interpolated from the orbit state vectors using `numpy.interp`.

For a spacecraft in low-Earth orbit that undergoes regular occultations, however, the angle to Earth's geometric center is not the most meaningful quantity. The relevant boundary is the Earth limb as seen from the spacecraft, since it is the limb that blocks the line of sight and marks the edge of the illuminated atmosphere responsible for stray-light events. Due to time constraints, angular computations to the nearest Earth limb are not yet integrated into the script. This is flagged as a limitation and left as a future extension (see Section 5.2).

3.4 Cosmographia Input Generation

Cosmographia is a 3D solar-system and spacecraft visualisation tool developed by NASA/JPL[¶]. It renders planets, moons, and spacecraft in a physically accurate inertial frame, supports custom spacecraft models and trajectories, and allows the user to navigate interactively in time and space. For a tool like `whereischeops`, which is concerned with the 3D orientation of a spacecraft body relative to celestial objects, Cosmographia provides a sanity check on the quaternion rotation logic and a visually intuitive complement to the angle plots.

Cosmographia comes with built-in 3D models for a range of NASA missions, allowing those spacecrafts to be loaded directly from the application's catalog. CHEOPS, as an ESA mission, is not among them. Loading it therefore requires sourcing a 3D model independently and constructing all catalog files from scratch. Loading CHEOPS into Cosmographia 4.2 requires four input files, each in a format specific to Cosmographia's internal data model, and none of which matches the native CHEOPS data products directly. A separate script, `generate_cosmographia_files.py`, performs all necessary conversions. Its implementation is detailed in Appendix F.

The spacecraft position and velocity must be provided as a plain-text table in Cosmographia's `.xyzv` format, with columns for time in TDB (Barycentric Dynamical Time) Julian Date followed by position (X, Y, Z) in km and velocity $(\dot{X}, \dot{Y}, \dot{Z})$ in km/s. The state vectors are already in this form, so the conversion consists of parsing the orbit file, transforming the UTC timestamps to TDB, and writing the table.

As for the attitude files, CHEOPS' orientation must be provided in Cosmographia's `.q` format, with TDB Julian Date followed by a unit quaternion in scalar-first order (q_w, q_x, q_y, q_z) representing the rotation from the inertial frame to the body frame. This convention is the conjugate of the PSE quaternion convention used in the FITS housekeeping files, which stores the body-to-inertial rotation in scalar-last order (q_x, q_y, q_z, q_w).

The spacecraft geometry is provided as a `.3ds` mesh file, a 3D Studio format that Cosmographia uses for spacecraft body rendering. The model currently renders as a black solid in Cosmographia, most likely because the original file is corrupted (see Section 5.1).

[¶]See: `https://naif.jpl.nasa.gov/naif/cosmographia.html`

Finally, Cosmographia loads the scene description from a JSON catalog file that specifies all scene objects, their data sources, display properties (label colors, trajectory arc duration and color, body-axis overlay size), and the time bounds of the visualization. The catalog file for this project, `cheops.json`, is described in Appendix F.

4 Results

Figure 4 shows the terminal output of a single-point orientation query issued at 2025-07-24T12:23:00 UTC. The tool interpolates the orbit trajectory to the request timestamp and reports the spacecraft position in the EME2000 reference frame. The tool also locates the nearest attitude measurement (0.22 seconds before the requested timestamp). The LoS-to-Sun angle of 115.07° lies at the declared Sun exclusion boundary. The radiator axis is at 86.00° from the Sun and 176.03° from Earth, meaning that the radiators are clear of both the Sun and Earth limb.

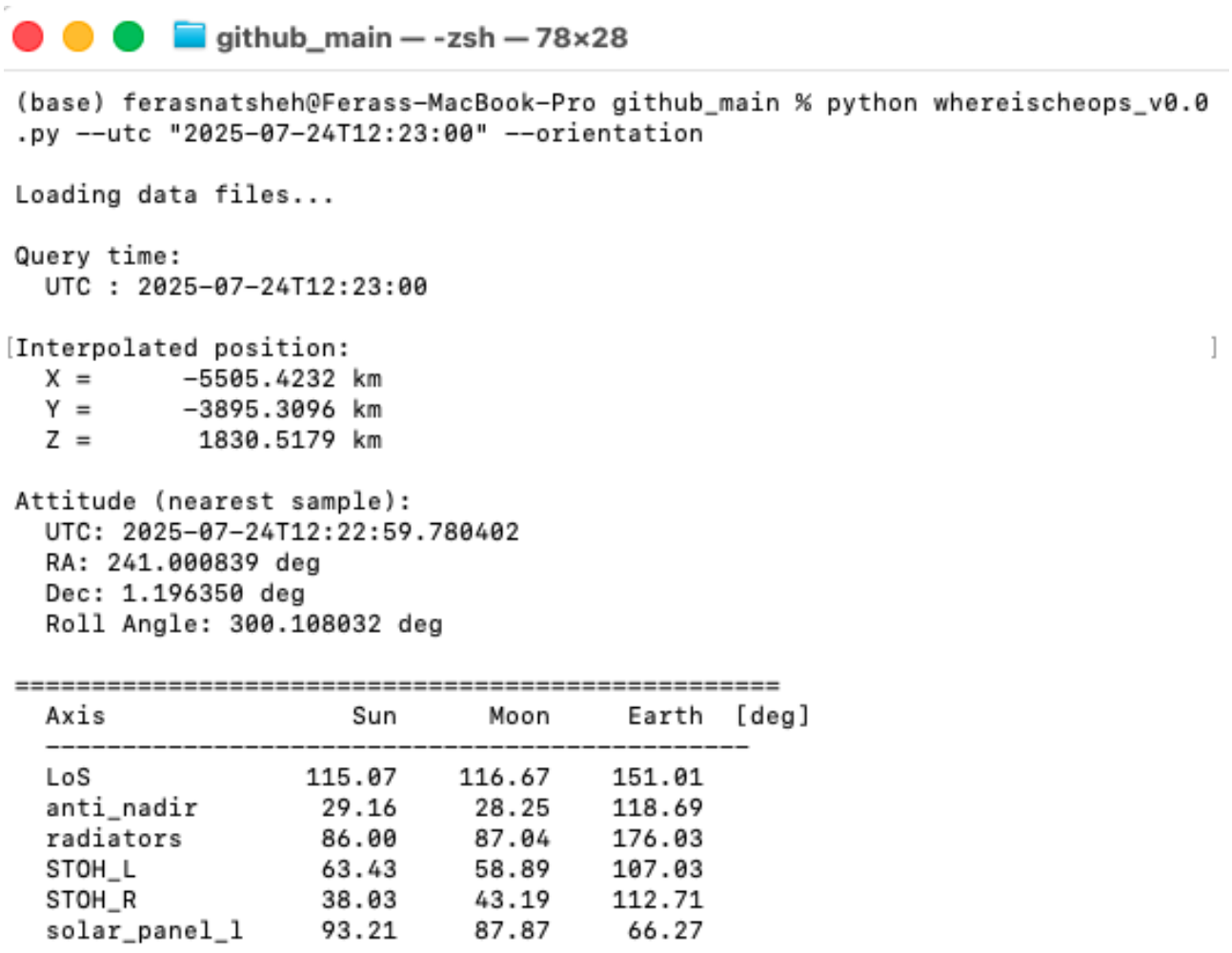
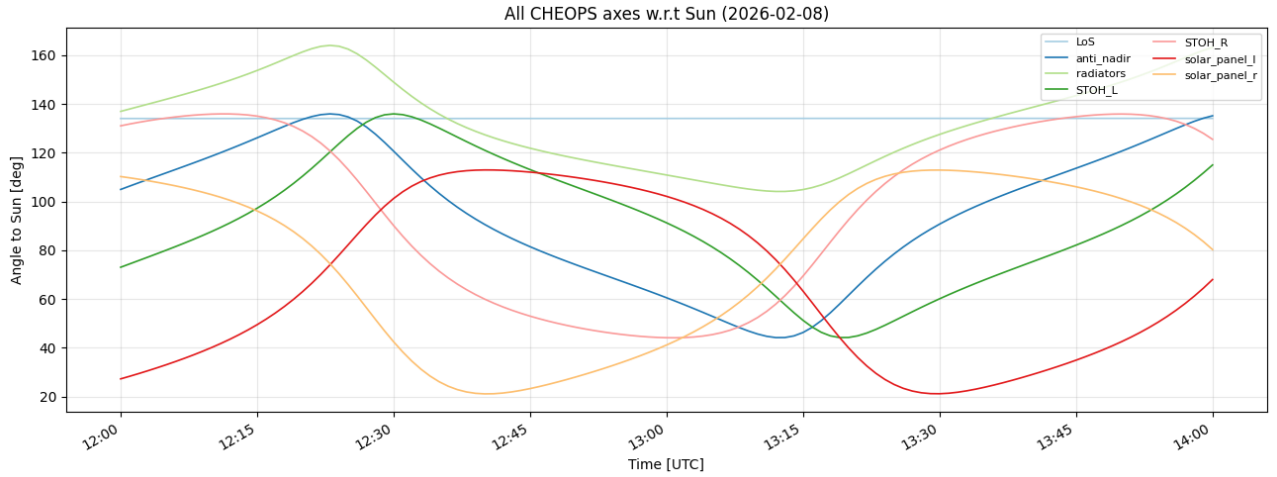


Figure 4: Terminal output of `whereischeops` in single-timestamp mode at 2025-07-24T12:23:00 UTC. The tool reports the interpolated spacecraft position, the nearest attitude measurement, and the angular separation of all body-frame axes to the Sun, Moon, and Earth center.

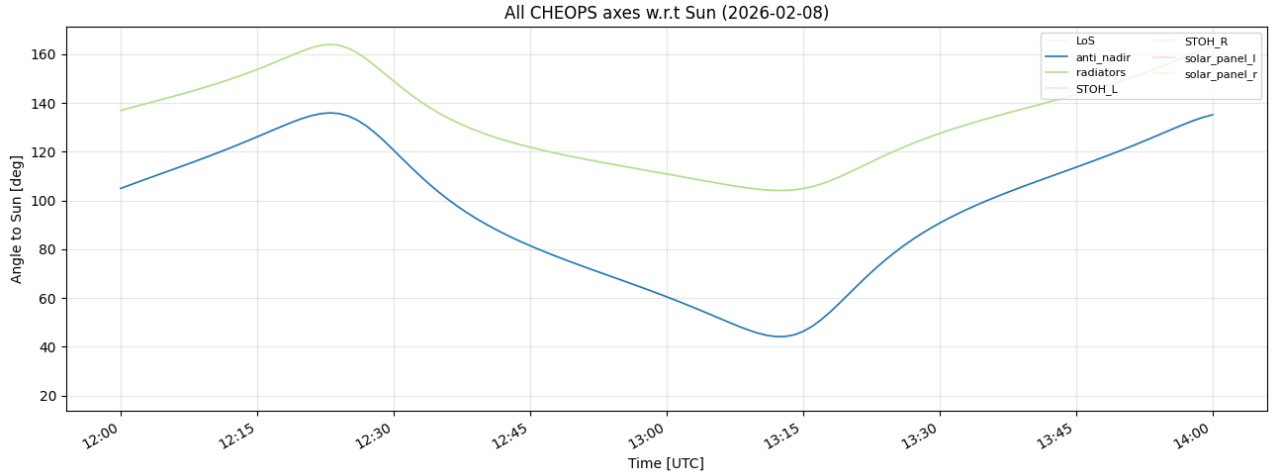
As for the time-range mode, Figures 5a and 5b illustrate the output for a two-hour window

on 2026-02-08, plotting the angle of every body axis to the Sun. The LoS angle in light blue (Figure 5a) is nearly flat at $\approx 133^\circ$ throughout the window because CHEOPS is staring at a fixed inertial target, the LoS direction is constant in the inertial frame and the slow apparent motion of the Sun over two hours produces a small change in this angle. On the other hand, all other body axes oscillate with an orbital period as the spacecraft rolls about the LoS to maintain semi-nadir-locking and solar panel illumination. This results in these axes sweeping through a wide range of Sun angles over each orbit. For example, the solar panel axes (red and orange) span roughly 20° to 115° within the two-hour window, confirming that the roll is actively keeping the panels in an orientation that is optimal for illumination.

While the number of simultaneously active lines makes it difficult to isolate a specific axis, the interactive legend filter, implemented via Matplotlib's `pick_event` callback, allows individual curves to be toggled without re-running the script. Figure 5b shows the same window after deactivating all other axes except the anti-nadir and the radiators, which reduces the visual clutter. Inspecting just these two axes shows that they are shifted by $\approx 20 - 60^\circ$ due to radiators being partially tilted in both $+X$ and $+Z$ axes.



(a) All body-frame axes rendered simultaneously.



(b) Same window after interactively toggling off all axes except anti_nadir and radiators.

Figure 5: Sun-angle diagnostic for a two-hour window on 2026-02-08, demonstrating the interactive legend filtering feature.

Figure 6 shows the three-panel attitude output for 2025-07-24. The RA and Dec panels confirm that the telescope pointing is locked to the target throughout the time window. The roll angle panel shows the non-linearity of the roll over each $\sim$ 99-minute orbit. Three complete cycles are visible over the four-hour window.

4.1 High-Jitter Visits

The pointing jitter is the instantaneous angular distance between the commanded and the measured telescope directions, and it is stored as a parameter in the pipeline quality report. When the jitter exceeds 2.5 arcseconds, it may impact the photometric data. The STOH-to-Sun angle is the primary geometric diagnostic used to investigate the source of pointing jitter. If the Sun enters the star-tracker field of view, scattered photons corrupt the attitude determination system and increase the pointing error. The `-plot sun` mode overlays both STOH-to-Sun angles with the jitter time series and shades in red every interval during which the pointing jitter exceeds the 2.5" critical threshold. Of course, this overlay is visible only when applicable.

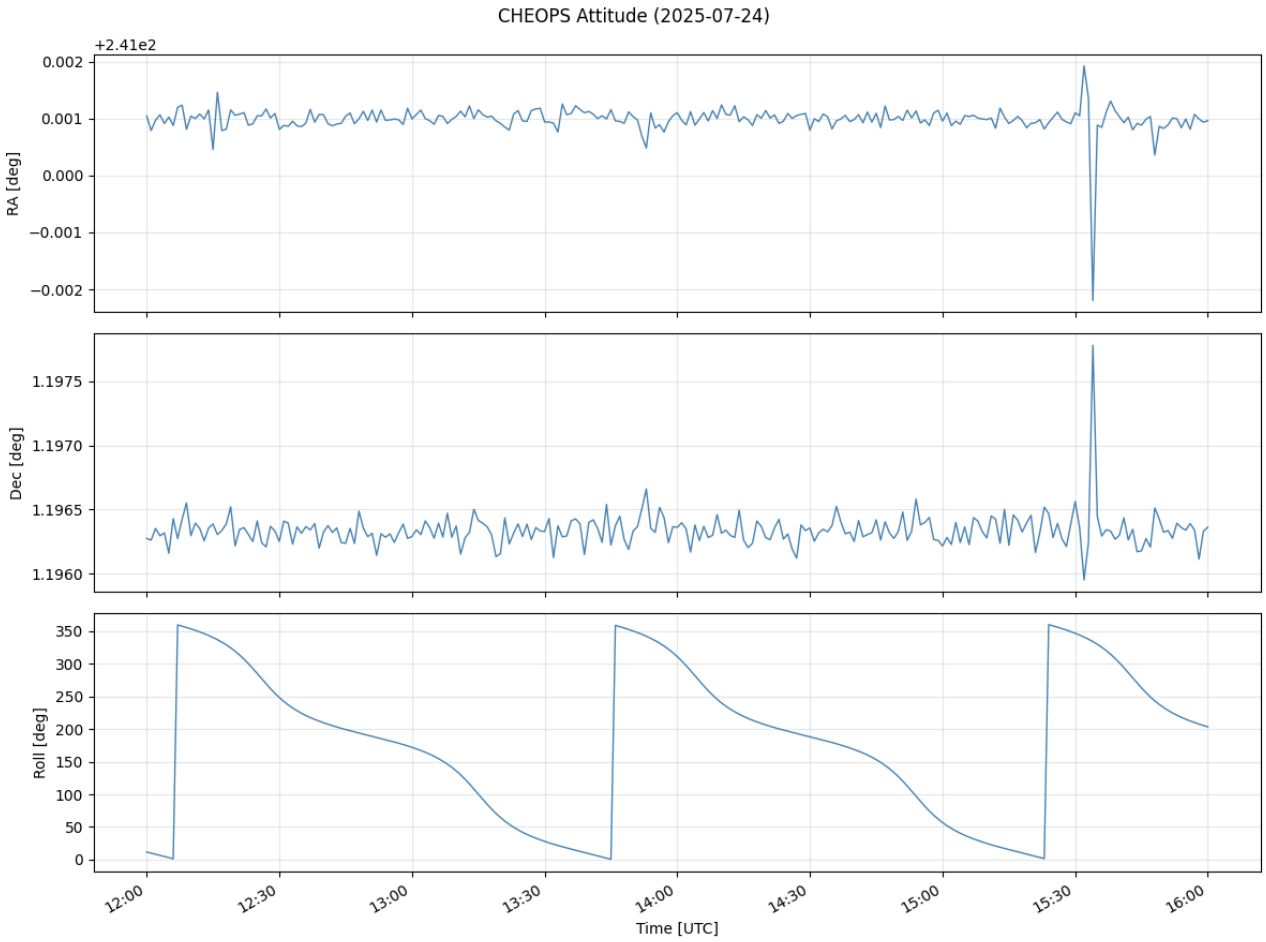


Figure 6: CHEOPS attitude time series for 2025-07-24, showing RA (top), Declination (middle), and roll angle (bottom) over a four-hour window.

The script searches for jitter data for the queried time range and returns the overlay plot when it finds relevant jitter data.

Figure 7 shows the Sun-angle diagnostic for the high-jitter visit of 2025-07-24, filtered to display only the two star-tracker heads. Both STOHs oscillate with the $\approx$ 99-minute orbital period as the spacecraft rolls about the LoS, but they reach their Sun-angle minima at slight different orbital phases, separated by $\sim 15 - 20$ minutes, due to their geometrical orientation (both STOHs are tilted 45° from the $+Z$ axis, so they are stationed 90° apart). The pointing-error time series in the lower panel shows that the jitter exceeds 2.5" exclusively within these windows during which the separation between the STOHs and the Sun is minimum. This suggests that the stray light from the Sun is blinding the STOHs, introducing imprecision in spacecraft's ability to point at the visit target.

Pointing jitter in CHEOPS could be due to various factors (e.g., thermally-driven instabilities). However, diagnosing the solar stray-light contamination of the star-tracker heads when both approach minimum Sun angle, is one of the main goals of this project, and is a direct test case for the validity of the `whereischeops` tool.

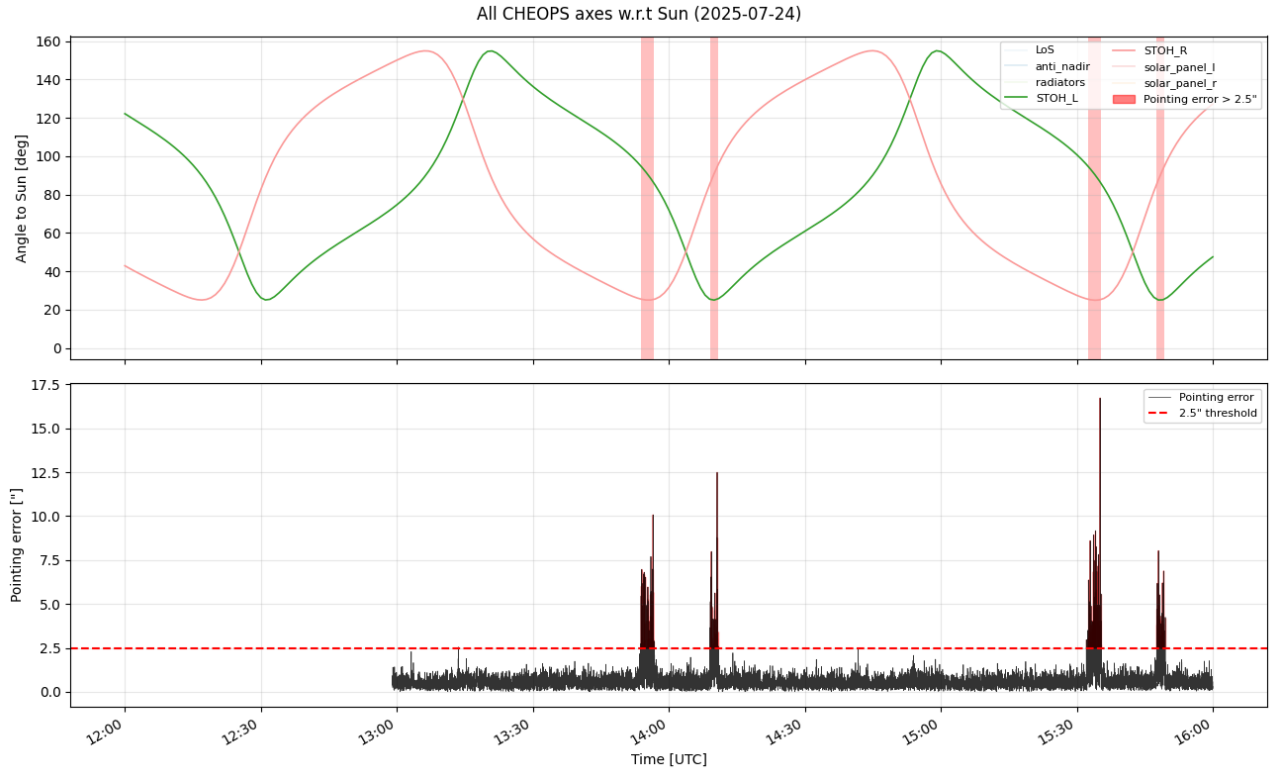


Figure 7: Sun-angle diagnostic for the high-jitter visit PR350112 (2025-07-24, 12:00-16:00 UTC). The figure is filtered to show only the two Star Tracker Optical Heads (STOHs) in the upper panel, where the red shading marks intervals where the pointing error exceeds 2.5". The lower panel shows the pointing error time series, where the red dashed line marks the critical jitter threshold of 2.5".

4.2 Anomalies

One of the primary motivations for developing `whereischeops` is to interpret the spacecraft geometry as a time series, which makes it straightforward to notice systematic deviations from expected behavior, and to identify anomalies beyond systematics errors. Figure 8 shows the angle between CHEOPS axes and the Earth center over the same visit window as above. The LoS (light blue) oscillates between $\approx 28°$ and $\approx 150°$ with the orbital period as expected. The anti-nadir ($+Z$, dark blue) and radiators (light green) also oscillate at the orbital period, but exhibit an anomaly at each of their maxima. Rather than a clean single peak, both curves display a brief plateau or double-peak structure near $\approx 176 - 178°$, in which the angle reaches a local maximum, undergoes a slight dip of a couple of degrees, and then recovers to a second local maximum before descending. The feature is present at every maximum in the four-orbit window and is more pronounced in the radiator curve.

This kind of behavior is not expected and neither does it match the physical consequences of rolling about the LoS. The origin of this behavior is not known. Nevertheless, it serves as a good test case for the `whereischeops` tool in identifying and allowing the CHEOPS SOC team to further investigate such anomalous cases.

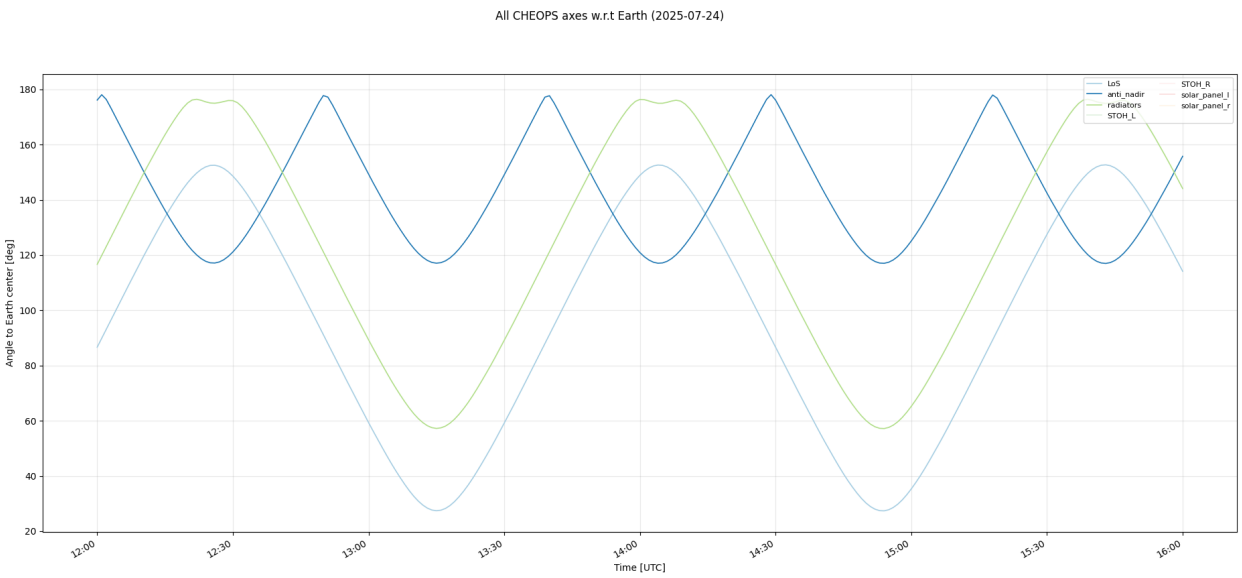


Figure 8: Angle between the center of Earth and LoS, anti-nadir, and radiators during a visit on 2025-07-24.

4.3 3D Orbit Visualization with Cosmographia

The main output of `whereischeops` is a set of angle time series, which are numerical curves that show how close each spacecraft body axis is to the Sun, Moon, or Earth as a function of time. A complementary representation is a full 3D visualization that lets the observer directly inspect the spacecraft's position in its orbit, the orientation of its body axes, and the geometry of occultations and body-frame sweeps in real time. Such visualizations can be implemented using Cosmographia.

The resulting visualization correctly renders the CHEOPS orbit with a trailing arc and the spacecraft 3D model. However, technical limitations should be noted (see Section 5.1). The visualization provides valuable geometric intuition such as orbital occultations visualized in 3-dimensional space, which complements the quantitative angle time series produced by the main tool. Figure 9 shows a snapshot of a preliminary visualization of CHEOPS in Cosmographia. Due to time constraints, debugging technical issues and rendering full-visit visualizations are not completed before the writing of this report, and are instead included as priority future extensions (see Section 5.2).

5 Limitations and Future Extensions

5.1 Limitation on the Current Implementation

As of writing this report, there are the following issues with the tool and the visualization:

- Earth-limb angle: Because CHEOPS is in low-Earth orbit, computing the spacecraft angles with respect to the nearest Earth limb is a much more useful diagnostic than the

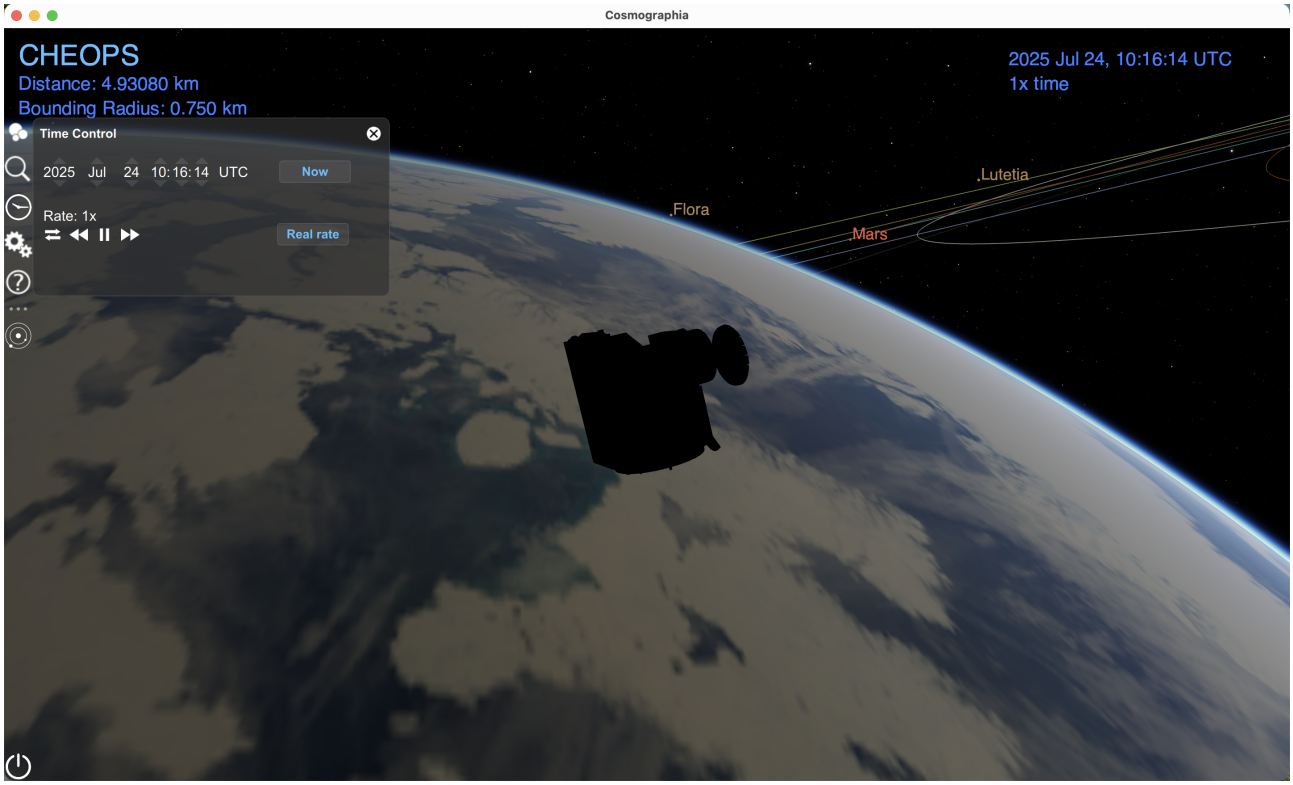


Figure 9: Cosmographia 4.2 visualization of CHEOPS on 2025-07-24.

angular separation to the Earth center, given that the angular diameter of the Earth from CHEOPS' 700km altitude covers a significant portion of the spacecraft field of view. Furthermore, Earth occultations and limb crossings account for a substantial fraction of each orbit, and are important factors to include in the tool to better understand pointing jitter. This feature will be implemented and pushed to the public GitHub repository.

- Black 3D model: The `CHEOPS_3D.3ds` file references two texture maps (`MappedTexture4.jpg` and `SolarPanels_FrontBackSides.jpg`) that are not included in the current file version. Without them, Cosmographia renders the geometry as a flat black mesh. To fix this, a new 3D model, with the missing files, should be recovered.

5.2 Future Extensions

The most impactful extension of `whereischeops` is scaling it to the full CHEOPS archival dataset. The current architecture already supports this. The data loader is parametrized by data directory and time window, so iterating over all visits requires only a wrapper function. The end goal of this project is for the CHEOPS SOC team to be able to use this tool on their archival and future data, so it remains the priority future extension.

A second targeted improvement is the addition of South Atlantic Anomaly passage intervals to the jitter diagnostics. SAA passages produce elevated particle flux that can independently trigger attitude disturbances. Flagging these intervals explicitly will help in understanding the root causes of the pointing jitter.

The Cosmographia visualizations can be expanded to realistically model target visits. The

quaternion data files include a pre-computed estimate of the trajectory and attitude of CHEOPS for a few visits in advance. This data can be scripted directly into Cosmographia to visualize in near real-time where CHEOPS is and what targets it is observing, and can be displayed on the observatory screen at the reception of the Geneva Observatory. This would allow visitors and researchers alike to see what CHEOPS is currently observing, where it is in orbit, and how its geometry relates to the Sun and Earth at any given moment.

6 Conclusion

`whereischeops_v0.0.py` provides an analysis chain from native CHEOPS data products to time-series geometric angles and interactive 3D visualizations. The tool performs attitude transformations and angle computations, and operates as a pipeline with minimal input requirements, making it scalable to archival datasets. The software, along with all relevant files and visualizations, is available at the project repository[‖] and documented in the Appendices below.

Declaration of Generative AI Use

In line with the University of Geneva Faculty of Science Directive on the Use of Generative Artificial Intelligence, I hereby declare that I have used a generative artificial intelligence tool (specifically Anthropic Claude) for the following:

- Partial code generation and debugging of the `whereischeops` Python script
- Assistance with scientific problem-solving, including the derivation and debugging of the quaternion and file conventions for Cosmographia; and
- Editing this report, including linguistic corrections, rephrasing, and LaTeX debugging.

All AI-generated content was reviewed, verified, and revised by the author. The scientific judgments, validations, results, and conclusions are the author's own.

[‖] `https://github.com/ferasnatsheh/whereischeops`

References

Astropy Collaboration et al., 2013, , 558, A33

Astropy Collaboration et al., 2018, , 156, 123

Astropy Collaboration et al., 2022, , 935, 167

Benz W., et al., 2021, The CHEOPS mission (arXiv:2009.11633), doi:10.1007/s10686-020-09679-4

Feissel M., Mignard F., 1998, , 331, L33

Fortier A., et al., 2024, CHEOPS in-flight performance: A comprehensive look at the first 3.5 years of operations (arXiv:2406.01716), https://arxiv.org/abs/2406.01716

Harris C. R., et al., 2020, Nature, 585, 357

Kaplan G. H., 2006, The IAU Resolutions on Astronomical Reference Systems, Time Scales, and Earth Rotation Models (arXiv:astro-ph/0602086), https://arxiv.org/abs/astro-ph/0602086

Virtanen P., et al., 2020, Nature Methods, 17, 261

Technical Documentation

The appendices here are written as a technical user manual for the CHEOPS Science Operations Center (SOC) team and any user of the `whereischeops` tool. They explain the code structure, format, and utility in detail.

A Software Architecture & Design

A.1 Overview

The script is a single Python file that requires Python 3.9 or later, and the installation of all dependencies (e.g., `pip install numpy`).

The file is organized into eight sections: (1) imports and constants, (2) time-conversion helpers, (3) file-parsing helpers, (4) data loader, (5) physics functions, (6) jitter helpers, (7) plot functions and helpers, and (8) the `main()` function.

A.2 Constants

- `DATA_DIR = "..."` assumes that all relevant data files are in the same directory
- `INTERNAL_STEP_SEC = 60` this is the time grid resolution (60 seconds)

- `JITTER_THRESHOLD_ARCSEC = 2.5` the jitter error maximum critical threshold in arcseconds
- `BODY_AXES = ...` the unit vectors of each component of CHEOPS in the body reference frame
- `R_EARTH_KM = 6371.0` Radius of Earth in km
- `_COLORS` sets a default color palette
- `_STYLES` sets default line styles

A.3 Data Loader

`load_data(data_dir, jd_start, jd_end)` is the function that imports all relevant data files. It returns a dictionary with keys `"orbit"`, `"attitude"`, `"quat"`, `"meta"`, and optionally `"jitter"`

For the orbit files, it discovers all files matching `CH_FDS_ORBRES_OPER_*.xml` in `data_dir`. It extracts the creation timestamp, start epoch, and end epoch from each filename, discards files whose coverage does not overlap $[jd_start, jd_end]$, and retains only the most recent version where multiple files cover the same window. The remaining files are parsed with `xml.etree.ElementTree` and concatenated into a single time-sorted array.

As for the FITS files, attitude (`SCI_RAW_Attitude...`) and quaternion (`SCI_RAW_HkAsy...`) files are loaded by `glob` pattern and concatenated without filename-based time filtering. All data are retained and sorted by internal timestamp. MJD values from the quaternion file are converted to Julian Date by adding 2 400 000.5.

Program ID, visit ID, target name, target right ascension and declination are extracted from the primary FITS header of the first matched attitude file and stored in `data["meta"]`.

Finally, if a file matching `*PIP_REP_MultiParameters*activity*.fits` is found, the function loads rows where `NAME == 'pointing error'` and returns the corresponding time and value arrays clipped to the requested window. If no such file is present, `data["jitter"]` is `None` and the jitter overlay is not shown.

A.4 Plot Architecture & Interactive Legends

Each plot mode is a self-contained function that receives the pre-loaded `data` dictionary and a time grid. All multi-line plots include a clickable legend implemented through Matplotlib's `pick_event` mechanism. Each legend line is registered as a picker (`set_picker(True)`), and a callback toggles the visibility of the corresponding data line when clicked. This is especially useful when inspecting plots with eight concurrent axis curves, as the user can isolate individual axes of interest without regenerating the plot.

A.5 Single Timestamp vs. Time-Range

The two operating modes share the same data loader but diverge in what they compute. Single-point mode (`-utc` or `-jd`) interpolates the orbit to a single epoch and reports the nearest-quaternion attitude. Time-range mode (`-timerange -start ... -end ...`) calls `nearest_rotations` for the full grid, calls `angle_to_target` once per body axis, per celestial body, and outputs the selected plot function.

B Command-Line Interface

No package installation is required for `whereischeops_v0.0.py` as it runs directly as a Python file in the terminal. The script can be run by running `python whereischeops_v0.0.py <flags>` in a terminal window, after making sure the file is in the current directory (e.g., `cd Astro/CHEOPS`)

B.1 Flags & Arguments

Table 2: All command-line flags

Flag	Type	Description
`-utc "YYYY-MM-DDTHH:MM:SS"`	string	Single-point mode: query UTC epoch
`-jd <value>`	float	Single-point mode: query Julian Date
`-timerange`	flag	Enable time-range mode
`-start "YYYY-MM-DDTHH:MM:SS"`	string	Range start (required with `-timerange`)
`-end "YYYY-MM-DDTHH:MM:SS"`	string	Range end (required with `-timerange`)
`-plot <type>`	string	Plot to generate (see Table 3)
`-orientation`	flag	Print axis-to-target angle table (single-point)
`-data-dir <path>`	string	Directory containing data files (default: `DATA_DIR`)

Table 3: Available `-plot` arguments in time-range mode

Argument	Output
`roll_angle`	Roll angle vs. time
`attitude`	RA, Dec, and roll angle vs. time (3 panels)
`sun`	All axes to Sun; jitter overlay if available
`moon`	All axes to Moon
`earth`	All axes to Earth center and limb (2 panels)
`all`	4-panel overview: roll, Sun, Moon, Earth key axes

B.2 Examples

Single-point orientation query:

```
python whereischeops_v0.0.py \
    --utc "2025-07-24T15:30:00" \
    --orientation
```

This prints a table of body-axis angles to the Sun, Moon, and Earth center at the specified timestamp.

Time-range jitter diagnostic:

```
python whereischeops_v0.0.py \
    --timerange \
    --start "2025-07-24T12:57:47" \
    --end   "2025-07-25T00:00:00" \
    --plot sun
```

Earth-angle overview for a full visit:

```
python whereischeops_v0.0.py \
    --timerange \
    --start "2025-07-24T00:23:00" \
    --end   "2025-07-25T00:00:00" \
    --plot earth
```

C Data File Formats & Naming Conventions

C.1 FITS Files

FITS files follow:

CH_PR*XXXXXX*_TG*YYYYYY*_TU*timestamp*_*TYPE*_V*NNN*.fits

where the timestamp has the format YYYY-MM-DDTHH-MM-SS. Key product types are: SCI_RAW_Attitude, SCI_RAW_HkAsy30767, SCI_RAW_SubArray, and PIP_REP_MultiParameters.

C.2 XML Files

Orbit files follow:

CH_FDS_ORBRES_OPER_*creation*_*start*_*end*.xml

where all three timestamps use the compact format YYYYMMDDHHMMSS with no separators. The loader parses these with datetime.strptime(s, "%Y%m%d%H%M%S").

C.3 Time Systems

All internal arithmetic uses Julian Date in the UTC time scale. MJD values from the quaternion and quality-report files convert to JD by JD = MJD + 2 400 000.5. The Cosmographia conversion additionally requires TDB Julian Date, obtained via `astropy.time.Time(utc, scale='utc').tdb.jd`.

D Dependencies and Used Libraries

`scipy.spatial.transform.Rotation.from_quat` (Virtanen et al., 2020)
Constructs a `Rotation` object from a quaternion in scalar-last convention $[x, y, z, w]$. Accepts an $N \times 4$ array to create a stack of N simultaneous rotations. Used in `nearest_rotations()` to build the full rotation stack for a visit in a single call.

`scipy.spatial.transform.Rotation.apply`
Applies the stored rotations to a body-frame vector via the quaternion sandwich product $q\mathbf{v}q^{-1}$. When called on a stack of N rotations, returns an $N \times 3$ array of inertial-frame vectors. Used in `angle_to_target()`.

`numpy.einsum`
Einstein-notation tensor contraction. The expression `np.einsum('ij,ij->i', v, t)` computes the row-wise dot product $\sum_j v_{ij} t_{ij}$ for each row i of two $N \times 3$ matrices, yielding the N cosine values from which angles are obtained via `numpy.arccos`. This replaces `astropy.coordinates.SkyCoord.sepa` as a more concise way to computing angles between two Cartesian unit vectors.

`numpy.interp` (Harris et al., 2020)
One-dimensional piecewise-linear interpolation. Given sorted reference arrays (x_p, f_p), returns interpolated values at query points x. Used to resample orbit position and attitude components onto the uniform time grid.

`astropy.coordinates.get_sun` / `get_body` (Astropy Collaboration et al., 2013, 2018, 2022)
Return the position of a solar-system body at a given `Time` as a `SkyCoord`.Used to compute N positions in a single call. The resulting Cartesian coordinates are normalized to unit vectors.

`astropy.time.Time`
Unified time-format handler supporting JD, MJD, ISO, and ISOT. Used for all conversions between Julian Date (internal arithmetic) and UTC ISO strings (display and filename parsing), and for passing time objects to coordinate functions.

`astropy.io.fits`
Standard Python interface for FITS files. Loads three product types: attitude (`SCI_RAW_Attitude`), quaternion housekeeping (`SCI_RAW_HkAsy`), and pointing-quality report (`PIP_REP_MultiParameters`).

`argparse` (Python standard library)
Command-line argument parser. Defines and validates all flags (`-utc`, `-jd`, `-timerange`, `-start`, `-end`, `-plot`, `-orientation`, `-data-dir`), and auto-generates help text.

`xml.etree.ElementTree`
Standard-library XML parser. Used in `_load_xml()` to locate and extract epoch, position, and velocity fields from state vectors.

E Quaternion Convention & Pipeline

E.1 FITS Convention

The PSE quaternion $\mathbf{q}_{\text{FITS}} = (q_x, q_y, q_z, q_w)$ (columns `PSE_quaternion_x/y/z/scal`) represents the rotation that maps a vector from the spacecraft body frame to the inertial (EME2000) frame. This is body-to-inertial in scalar-last order, which matches the convention expected by `scipy.spatial.transform.Rotation.from_quat()` directly. No reordering or sign change is needed for `whereischeops`.

E.2 Cosmographia Convention

Cosmographia's `.q` format expects the rotation that maps vectors from the inertial frame to the body frame (the inverse of the FITS convention), with the scalar component first: (q_w, q_x, q_y, q_z).

For a unit quaternion, the inverse equals the conjugate: $\mathbf{q}^{-1} = (q_w, -q_x, -q_y, -q_z)$. Therefore, each row of the `.q` file is:

$$(q_w,\ -q_x,\ -q_y,\ -q_z) \quad \text{from FITS columns} \quad (\text{scal},\ -x,\ -y,\ -z). \tag{4}$$

The `fits_to_q()` function in `generate_cosmographia_files.py` implements this transformation.

F Cosmographia Visualization Guide

F.1 Overview

Cosmographia 4.2 renders spacecraft trajectories and approximate attitudes from four input files: a `.xyzv` trajectory file, a `.q` attitude file, a .3ds 3D model file, and a JSON catalog file.

F.2 Requirements

- Cosmographia 4.2 (available from the ESA Virtual Space Exploration Gateway or NASA/JPL)
- Python 3.9+ with `numpy`, `astropy`, and `astropy.io.fits`

- XML orbit file for the visit
- `SCI_RAW_HkAsy...` quaternion FITS for the visit
- `CHEOPS_3D.3ds` spacecraft model

F.3 File Converter

The script `generate_cosmographia_files.py` in the `cosmographia/` directory performs all three conversions. Edit the paths at the bottom of the file and run:

```
cd cosmographia/
python generate_cosmographia_files.py
```

This produces `cheops_trajectory.xyzv`, `cheops_attitude.q`, and `cheops.json`.

The `generate_cosmographia_files.py` contains: **`xml_to_xyzv(xml_path, out_path)`**: Parses the OEM XML file and writes one line per state vector in the format:

```
<TDB_JD> <X> <Y> <Z> <VX> <VY> <VZ>
```

with positions in kilometres and velocities in km/s. The epoch is converted from UTC ISOT to TDB Julian Date via `astropy.time.Time`.

`fits_to_q(fits_path, out_path)`: Reads the `SCI_RAW_HkAsy30767` extension and writes one line per quaternion sample in the format:

```
<TDB_JD> <qw> <qx> <qy> <qz>
```

applying the conjugate transformation of equation (4). The MJD timestamp is converted to TDB JD via `astropy.time.Time`.

`write_json(out_path, start_utc, end_utc)`: Writes the Cosmographia catalog JSON. The `startTime` and `endTime` fields must bracket the data in the trajectory file. Edit `start_utc` and `end_utc` to match the visit.

F.4 Loading into Cosmographia

1. Open Cosmographia 4.2.
2. Select **File** → **Open Catalog** and navigate to `cheops.json`.
3. Use the timeline at the bottom to navigate to the visit window.
4. The CHEOPS spacecraft should appear orbiting Earth. If it does not, verify that the simulation time is included in `startTime`/`endTime` in the JSON file.